Next-U Group Education

RAPPORT DE PROJET
CI/CD - API REST To-Do

Développé par
ALBAN Mathis - MASSACRY Audran - REY Florian

Année universitaire 2025 - 2026

SOMMAIRE

SOMMAIRE **2**
Ⅰ / PRÉSENTATION **3**
1. Objectif principal 3
2. Stack technique 3
Ⅱ / Architecture et conception **5**
1. Architecture et modèles de données 5
1.1. Structure du projet 5
1.2. Routes exposées 5
1.3. Modèle de données (Prisma) 6
2. Conteneurisation avec Docker 6
Ⅲ / Pipeline CI/CD **7**
Ⅳ / Démo **8**
1. Vérification de l'état du serveur 8
2. Liste toutes les tâches 8
3. Récupération d'une tâche 8
4. Création d'une tâche 8
5. Mise à jour d'une tâche 9
6. Suppression d'une tâche 9

I / PRÉSENTATION

1. Objectif principal

Ce projet a pour but la création d'une API REST de gestion de tâches (To-Do) avec persistance en base de données, conteneurisation avec Docker et mise en place d'un pipeline CI/CD. A celà s'ajoute un linter et un scan de sécurité pour garantir la qualité du code.

2. Stack technique

Pour mener à bien ce projet, nous avons choisi un environnement de travail autour de Node.js express et Prisma comme ORM.

Technologie	Rôle	Version
Node.js	Environnement d'exécution côté serveur	24 (LTS)
TypeScript	Typage statique	5.9.3
Express.js	Framework HTTP pour l'API REST	^5.2
Prisma	ORM	^7.4
PostgreSQL	Base de données relationnelle	15
Jest	Tests unitaires et d'intégration	29.7.0
ESLint	Analyse statique du code	^10.0.2
Docker et Docker Compose	Conteneurisation de l'app	–
Github Actions	Pipeline CI/CD	–

Ces choix architecturaux repose sur plusieurs points :

- **TypeScript et Express** : Avec Node.js, Express permet une grande flexibilité pour développer des API légères rapidement. TypeScript a été ajouté afin de sécuriser le code en réduisant les erreurs à l'exécution et en réduisant la maintenabilité du code sur le long terme.
- **Prisma et PostgreSQL** : PostgreSQL est une base de données relationnelle robuste. Pour l'interaction, nous avons choisi l'ORM Prisma : il simplifie la gestion des migrations et la sécurité de typage. Cet ORM fonctionne très bien avec TypeScript.
- **Jest et Supertest** : Deux packages pour tester l'API Node.js. Jest s'occupe des exécutions et assertions, tandis que Supertest permet de simuler les requêtes HTTP sans exposer l'application sur un vrai réseau.

- **Docker et Github Actions** : Docker garantit un environnement strict identique sur l'ensemble des machines. Github Actions, quant à lui, est parfait pour son intégration native avec le repo Github afin de pouvoir tester notre code.

Ⅱ / Architecture et conception

1. Architecture et modèles de données

1.1. Structure du projet

L'application a été structurée selon une architecture en couches afin de séparer les responsabilités.

```
src/
├── index.ts                    <- Point d'entré (listen sur PORT)
├── lib/
│   ├── app.ts                  <- Instance Express + middlewares
│   └── prisma.ts               <- Instance Prisma Client (singleton)
├── routes/
│   ├── index.router.ts         <- Routeur principal
│   ├── health.router.ts        <- Route GET /health
│   └── tasks.router.ts         <- Routes CRUD /tasks
├── controllers/
│   ├── health.controller.ts    <- Logique healthcheck (ping DB)
│   └── tasks.controller.ts     <- Logique CRUD pour task
└── responses/
    ├── success.response.ts     <- { success: true, data }
    └── error.response.ts       <- { success: false, error }
```

Le point d'entrée (index.ts) se charge uniquement de l'écoute du serveur. L'instance Express et les middlewares (CORS, Morgan pour les logs, parsing JSON) sont isolés dans le dossier lib/. Le routage est géré dans le répertoire routes/, qui délègue ensuite la logique métier aux fichiers du dossier controllers/. Les réponses HTTP sont standardisées via des utilitaires dédiés dans responses/ afin d'assurer une structure uniforme pour le client frontend.

1.2. Routes exposées

Au total, 6 routes sont accessibles par l'utilisateur :

- GET /health : Permet de vérifier l'état du serveur et la connexion à la base de données.
- GET /tasks et GET /tasks/:id : Permet de récupérer l'ensemble des tâches ou une tâche précise via son id.
- POST /tasks : Crée une nouvelle tâche.
- PUT /tasks/:id : Met à jour une tâche existante via son id.
- DELETE /tasks/:id : Supprime une tâche existante via son id.

1.3. Modèle de données (Prisma)

Le modèle de données est composé d'une unique table nommée "Task". Elle est composée des éléments suivants :

```
┌──────────────────────────────────────────┐
│                   Task                   │
├────────────────┬─────────────────────────┤
│ id             │ Int (PK, auto-increment) │
│ title          │ String                   │
│ description    │ String                   │
│ completed      │ Boolean (défaut: false)  │
│ dueDate        │ DateTime (défaut: now())  │
│ createdAt      │ DateTime (défaut: now())  │
│ updateAt       │ DateTime (@updatedAt)    │
└────────────────┴─────────────────────────┘
```

2. Conteneurisation avec Docker

Notre API repose sur un environnement Docker et Docker Compose. Nous avons deux services principaux :

- **Dockerfile** :
  - Compilation TypeScript et génération du client Prisma
  - Image de production allégée (utilisation du dossier dist) et exécution automatique des migrations.
- **Docker Compose** :
  - Service **postgres** : Persistance des données et healthcheck toutes les 5 secondes.
  - Service **server** : Démarre une fois la base de données opérationnelle.

Ⅲ / Pipeline CI/CD

Le pipeline garantit la qualité et le déploiement continu du code. A chaque push ou pull request sur la branche principale, le pipeline se déclenche automatiquement. Il est composé de 5 jobs :

- **Étape 1 : Vérification Statique (lint)** : Une vérification statique du code est effectuée. Tout échec à cette étape bloque l'exécution du reste du pipeline.
- **Étape 2 (en parallèle) : Préparation de l'environnement** :
    - **Build** : Génération du client Prisma et compilation du code.
    - **Tests** : Création d'une base de données PostgreSQL éphémère, application des migrations et exécution des tests unitaires/d'intégration.
    - **Scan de sécurité** : Analyse des fichiers pour détecter les vulnérabilités de niveau critique ou élevé.
- **Étape 3 : Déploiement de l'image (docker build et push)** : Cette étape est conditionnelle et ne s'exécute qu'après le succès du build et des tests. Elle consiste à se connecter à GitHub et à publier l'image Docker sur le Github Container Registry.

Ⅳ / Démo

1. Vérification de l'état du serveur

Requête

```
GET http://localhost:3000/health
```

Réponse attendue - 200 OK

```
{
  "status": "UP",
  "database": "CONNECTED",
  "timestamp": "2026-03-07T11:33:11.711Z"
}
```

2. Liste toutes les tâches

Requête

```
GET http://localhost:3000/tasks
```

Réponse attendue - 200 OK

```
{
    "success": true,
    "data": [
        {
            "id": 1,
            "title": "Create a new task",
            "description": "Description de la nouvelle tâche",
            "completed": false,
            "dueDate": "2023-12-31T23:59:59.000Z",
            "createdAt": "2026-03-07T11:30:25.463Z",
            "updateAt": "2026-03-07T11:30:25.463Z"
        }
    ]
}
```

3. Récupération d'une tâche

Requête

```
GET http://localhost:3000/tasks/:id
```

Réponse attendue - 200 OK

```
{
    "success": true,
    "data": {
        "id": 1,
        "title": "Create a new task",
        "description": "Description de la nouvelle tâche",
        "completed": false,
        "dueDate": "2023-12-31T23:59:59.000Z",
        "createdAt": "2026-03-07T11:30:25.463Z",
        "updateAt": "2026-03-07T11:30:25.463Z"
    }
}
```

4. Création d'une tâche

Requête

```
POST http://localhost:3000/tasks
Content-Type: application/json
```

```
{
   "title": "Create a new task",
   "description": "Description de la nouvelle tâche",
   "dueDate": "2023-12-31T23:59:59Z"
}
```

Réponse attendue - 201 Created

```
{
    "success": true,
    "data": {
        "id": 1,
        "title": "Create a new task",
        "description": "Description de la nouvelle tâche",
        "completed": false,
        "dueDate": "2023-12-31T23:59:59.000Z",
        "createdAt": "2026-03-07T11:30:25.463Z",
        "updateAt": "2026-03-07T11:30:25.463Z"
    }
}
```

5. Mise à jour d'une tâche

Requête

```
PUT http://localhost:3000/tasks/:id
Content-Type: application/json
```

```
{
   "title": "Test test",
   "description": "Updated",
   "dueDate": "2026-02-01T23:59:59Z",
   "completed": true
}
```

Réponse attendue - 204 No content

6. Suppression d’une tâche

Requête

```
DELETE http://localhost:3000/tasks/:id
```

Réponse attendue - 204 No content